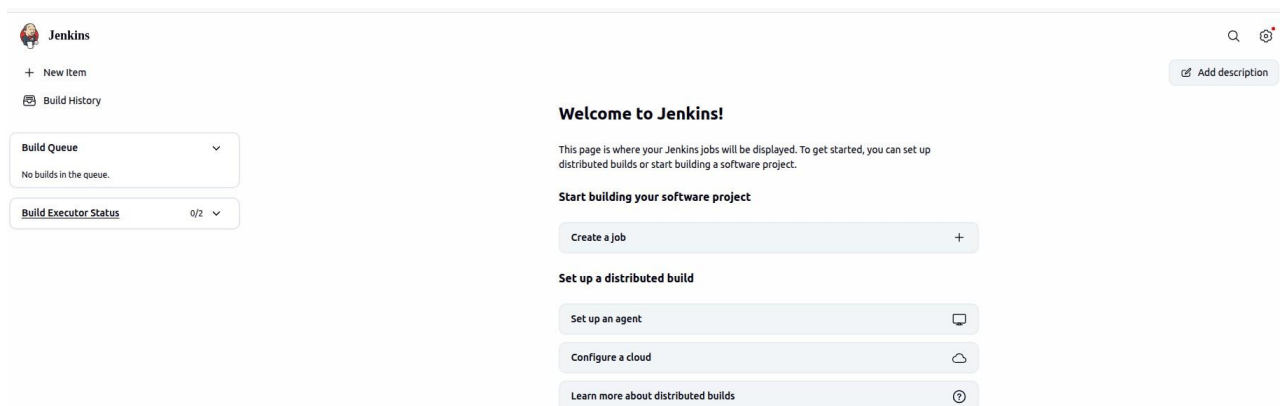


Program: Jenkins Pipeline Setup including plugins installation and tools configuration for Simple Maven

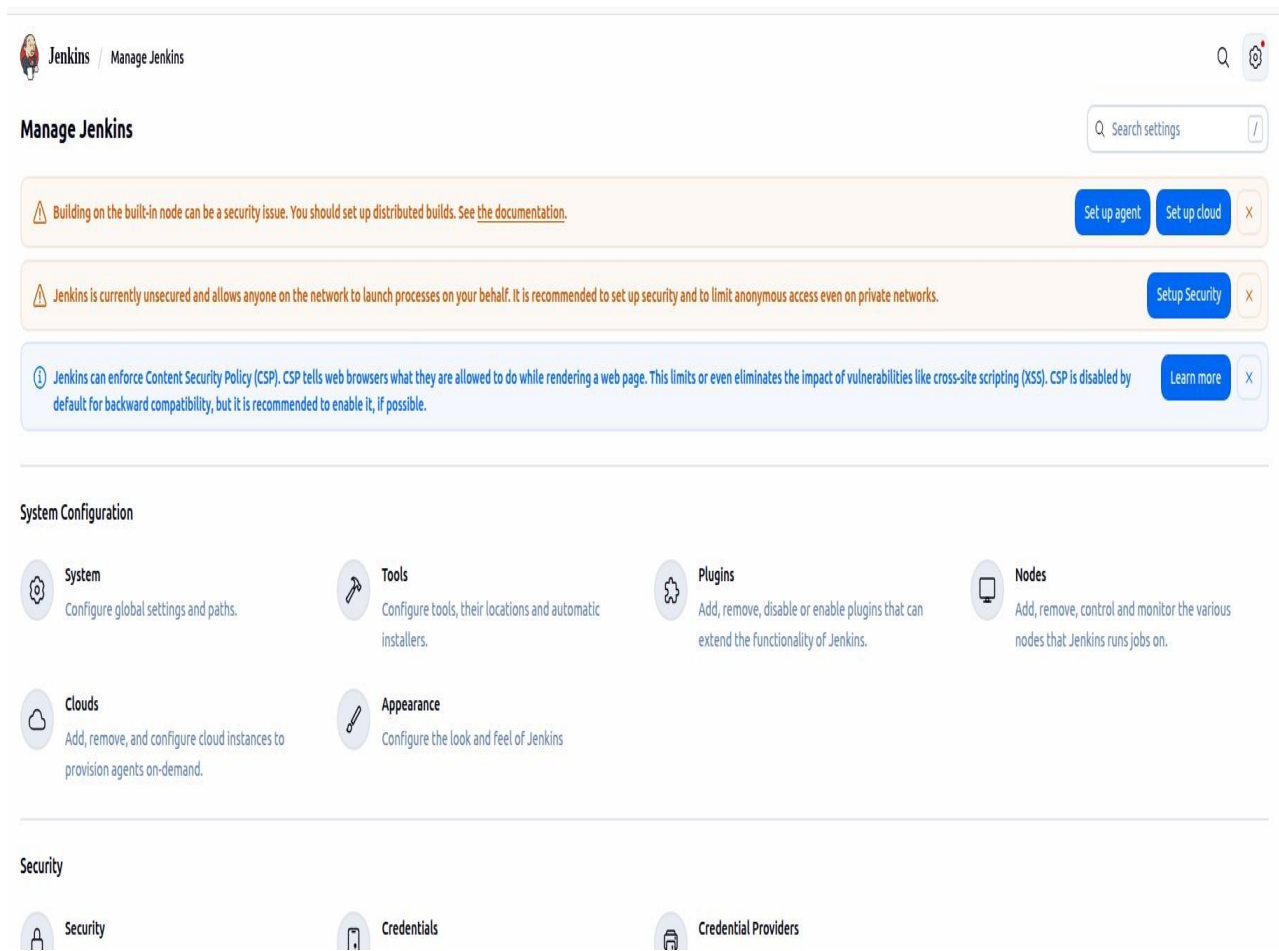
Jenkins is a popular open-source automation server used for continuous integration and continuous delivery (CI/CD). It enables developers to automate the process of building, testing, and deploying applications efficiently. Jenkins pipelines allow users to define the entire workflow of a project as code using a file called a Jenkinsfile, which ensures consistency and repeatability in the build process.

A Jenkins pipeline is divided into multiple stages such as checkout, where the source code is retrieved from a version control system; build, where the application is compiled; test, where automated tests are executed; and deployment or run, where the application is executed or deployed. These stages help in organizing the workflow and identifying issues at different phases of development.

Step 1: Login to Jenkins



Step 2 : Go to Manage Jenkins



Step 3: Click on plugins and install the required plugins

Navigate to Manage Jenkins → Plugins → Available Plugins.

1. Pipeline Stage view
2. Github Integration
3. Maven
4. Gradle



Plugins

📄 Updates

26

📁 Available plugins

⚙️ Installed plugins

⚙️ Advanced settings

☰ Download progress

Download progress

Preparation

- Checking internet connectivity
- Checking update center connectivity
- Success

Pipeline Graph Analysis ✓ Success

Pipeline: REST API ✓ Success

Pipeline: Stage View ✓ Success

Loading plugin extensions ✓ Success

→ [Go back to the top page](#)
(you can start using the installed plugins right away)

→ ☐ Restart Jenkins when installation is complete and no jobs are running



Plugins

📄 Updates

26

📁 Available plugins

⚙️ Installed plugins

⚙️ Advanced settings

☰ Download progress

Download progress

Preparation

- Checking internet connectivity
- Checking update center connectivity
- Success

Pipeline Graph Analysis ✓ Success

Pipeline: REST API ✓ Success

Pipeline: Stage View ✓ Success

Loading plugin extensions ✓ Success

GitHub Integration ✓ Success

Loading plugin extensions ✓ Success

Dev Tools Symbols API ✓ Success

Javadoc ✓ Success

JSch dependency ✓ Success

Maven Integration ✓ Success

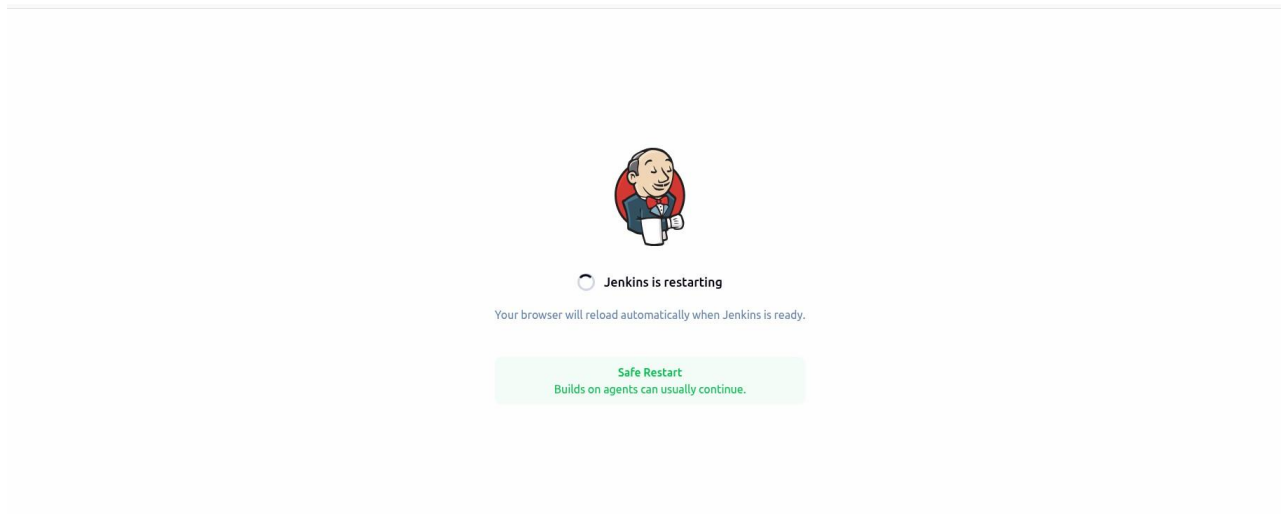
Loading plugin extensions ✓ Success

Restarting Jenkins 🔄 Running

→ [Go back to the top page](#)
(you can start using the installed plugins right away)

→ ☒ Restart Jenkins when installation is complete and no jobs are running

Step 4: After verifying installation restart Jenkins



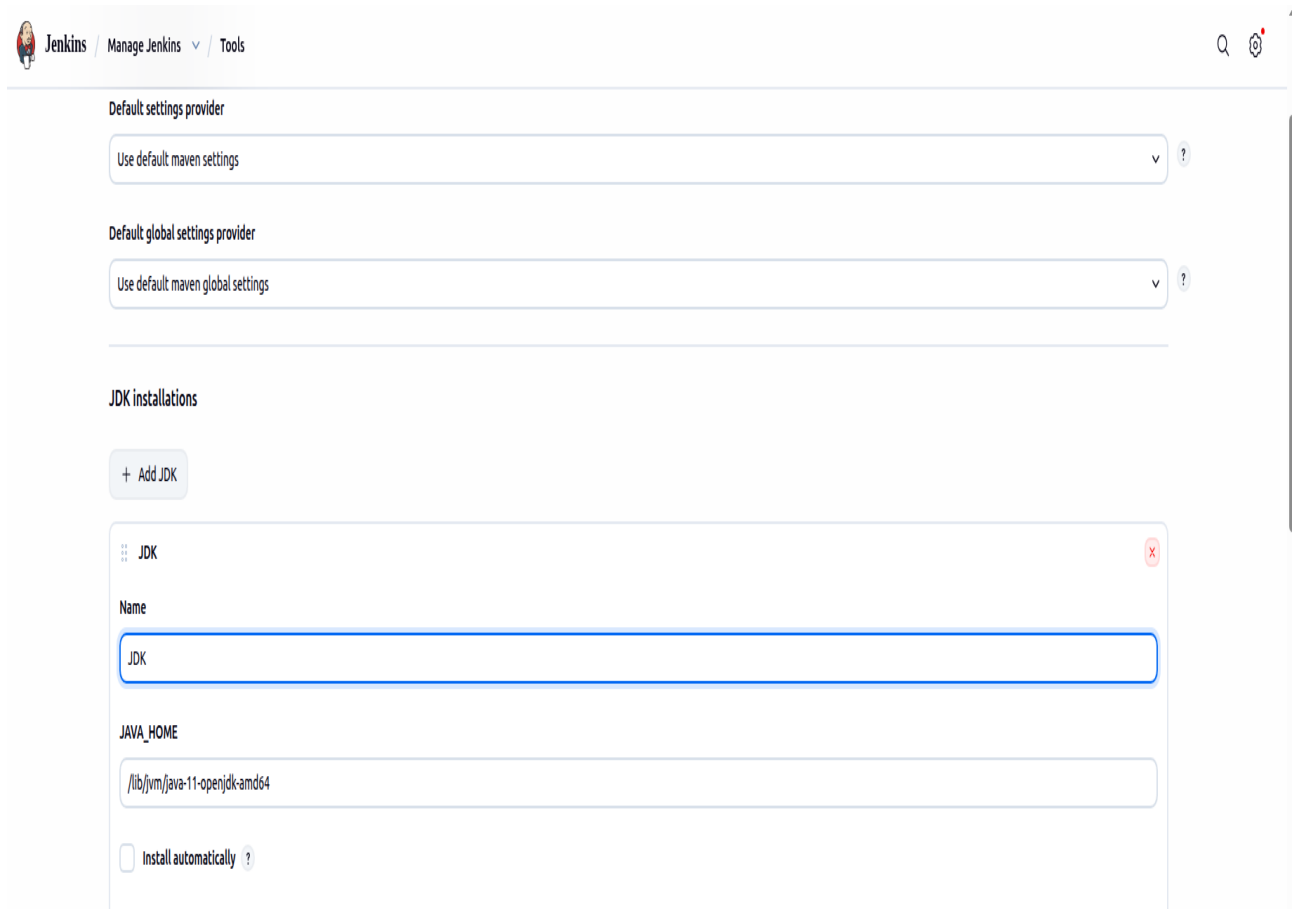
Step 5: Configure Tools

Go to Manage Jenkins → Tools.

Scroll

down and click Add JDK.

Enter JDK name and path (Computer → lib → jvm → jdk).



Enter the name and address The address of the JDK can be found in your files -> other location -> computer -> lib -> jvm -> jdk

Step 7: Configure Build Tools Find installation paths for Git, Maven, and Gradle using terminal.

Enter these paths in Jenkins and click Apply and Save.

```
rakshith@Ubuntu22:~$ which git
/usr/bin/git
rakshith@Ubuntu22:~$ which mvn
/usr/bin/mvn
rakshith@Ubuntu22:~$ which gradle
/usr/bin/gradle
rakshith@Ubuntu22:~$
```

Jenkins / Manage Jenkins / Tools

Git installations

Git

Name

Git

Path to Git executable ?

/usr/bin/git

☐ Install automatically ?

Jenkins Manage Jenkins Tools

Global Tool Configuration

+ Add Gradle

Gradle

name ?
Gradle

GRADLE_HOME ?
/usr/bin/gradle

⚠ /usr/bin/gradle is not a directory on the Jenkins controller (but perhaps it exists on some agents)

☐ Install automatically ?

Step 8: Add GitHub Credentials

Go to Manage Jenkins → Credentials

Jenkins Manage Jenkins Credentials

Credentials

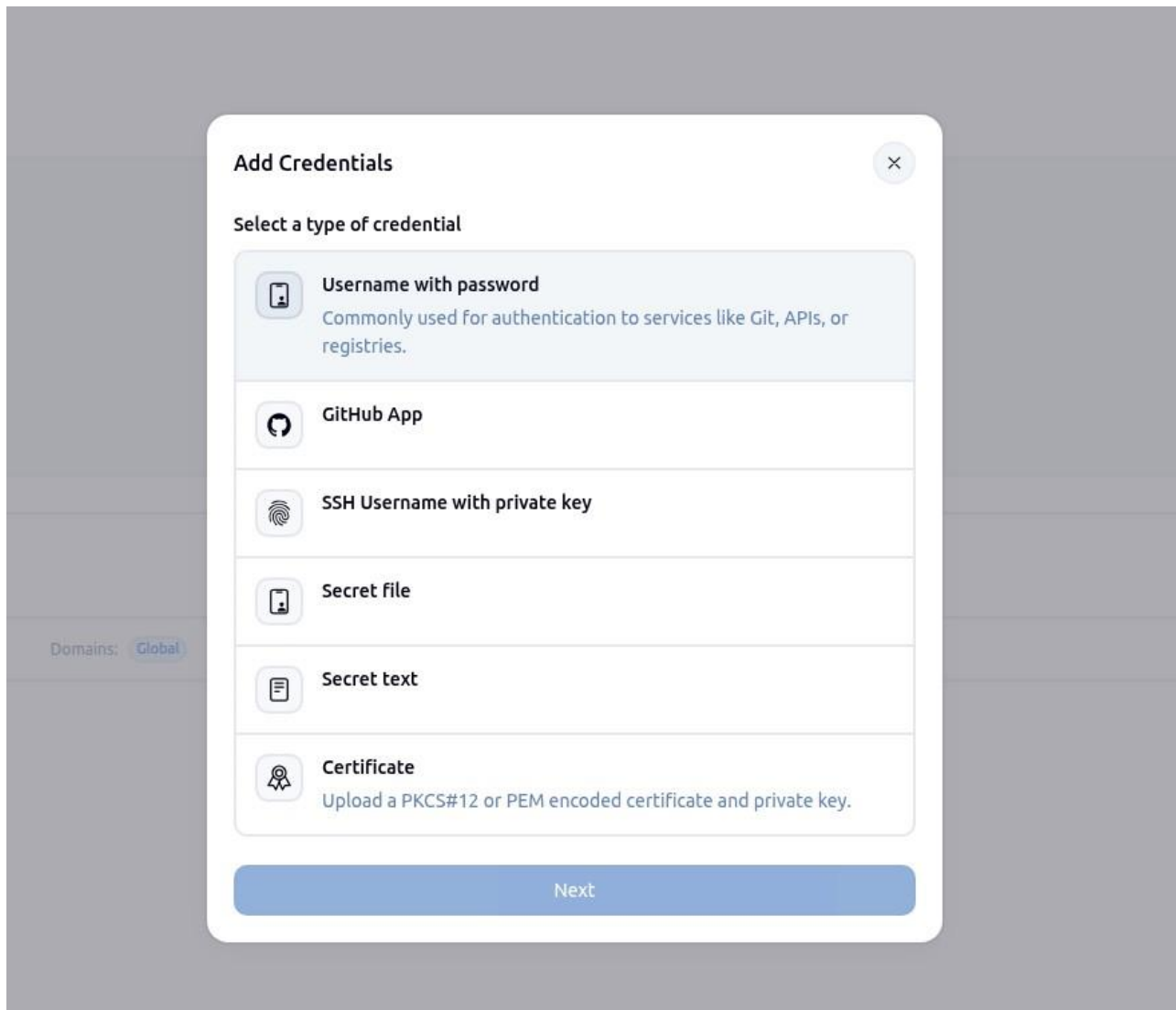
There are no credentials

+ Add Credentials

Stores scoped to Jenkins

System Domains: Global

Add Username with Password and click on next



Enter GitHub credentials and description as 'GitHub Credentials'.

<

Add Username with password

×

Scope ?

Global (Jenkins, nodes, items, all child items, etc) ▾

Username

rakshith31-hub

☒ Treat username as secret ?

Password

.....

ID ?

Description ?

GitHub Credential

Create

Verify if we have successfully created the Credential



Jenkins / Manage Jenkins ▾ / Credentials

Credentials

+ Add Credentials



GitHubCredentials

System · Global · 9767deec-da9c-4e2d-bd2a-df85ffbddb9e · GitHubCredentials



Stores scoped to Jenkins



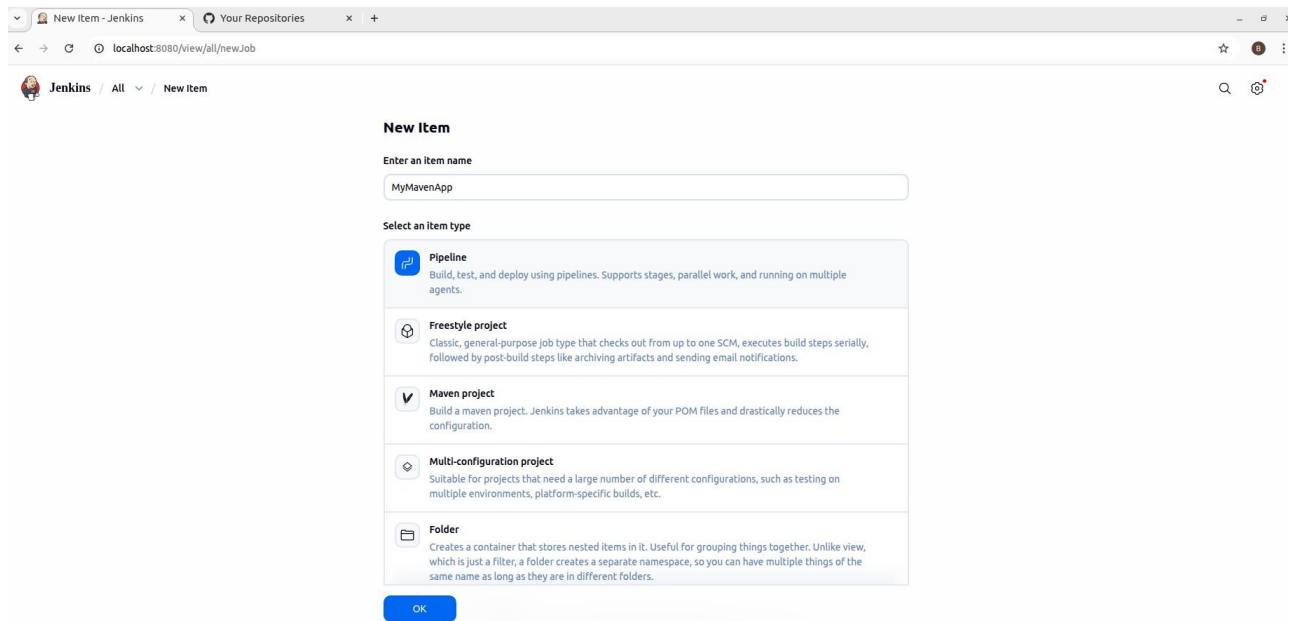
System

Domains: Global

Step 9: Create a New Pipeline Job

Go to Dashboard → + New Item.

Enter name, select Pipeline, and click OK.

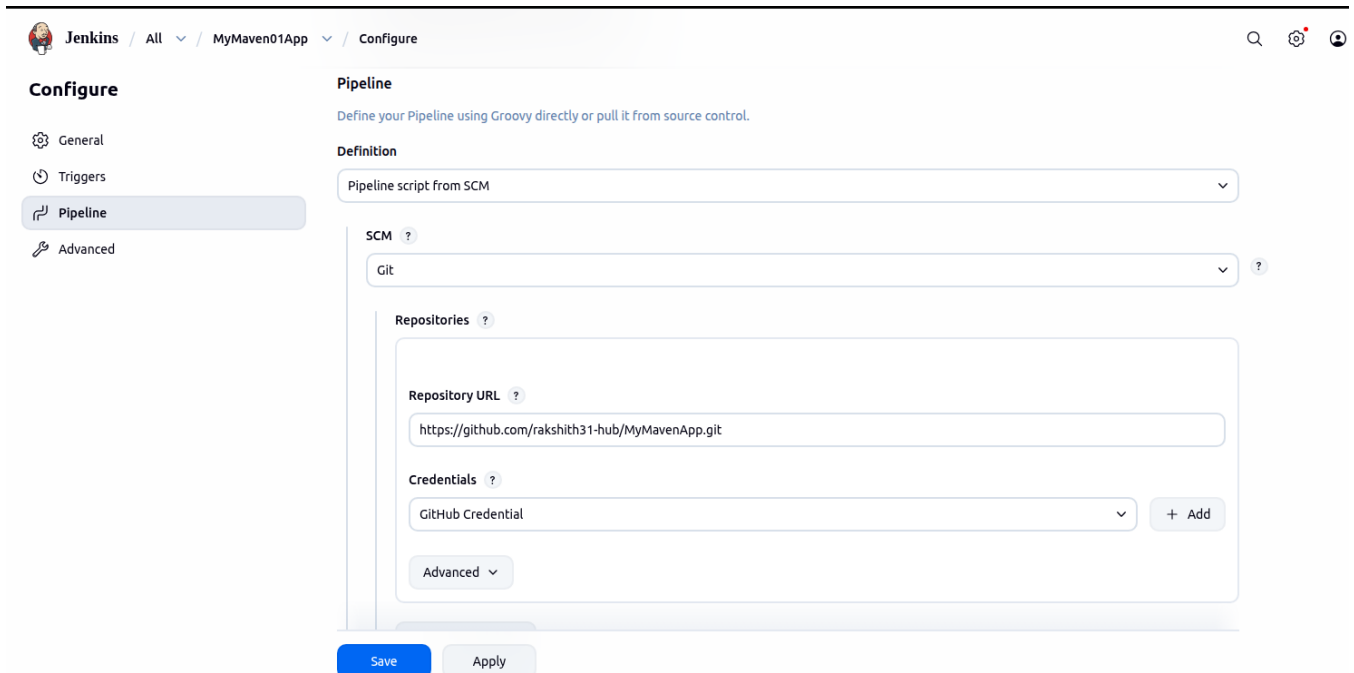


The screenshot shows the Jenkins 'New Item' configuration page. The browser address bar indicates the URL is `localhost:8080/view/all/newJob`. The page title is 'Jenkins / All / New Item'. The 'New Item' section has a text input for 'Enter an item name' with the value 'MyMavenApp'. Below this is a 'Select an item type' section with five options: 'Pipeline' (selected), 'Freestyle project', 'Maven project', 'Multi-configuration project', and 'Folder'. Each option has a brief description. At the bottom of the 'Select an item type' section is a blue 'OK' button.

Step 10: Configure Pipeline from SCM

Scroll down and select Pipeline script from SCM.

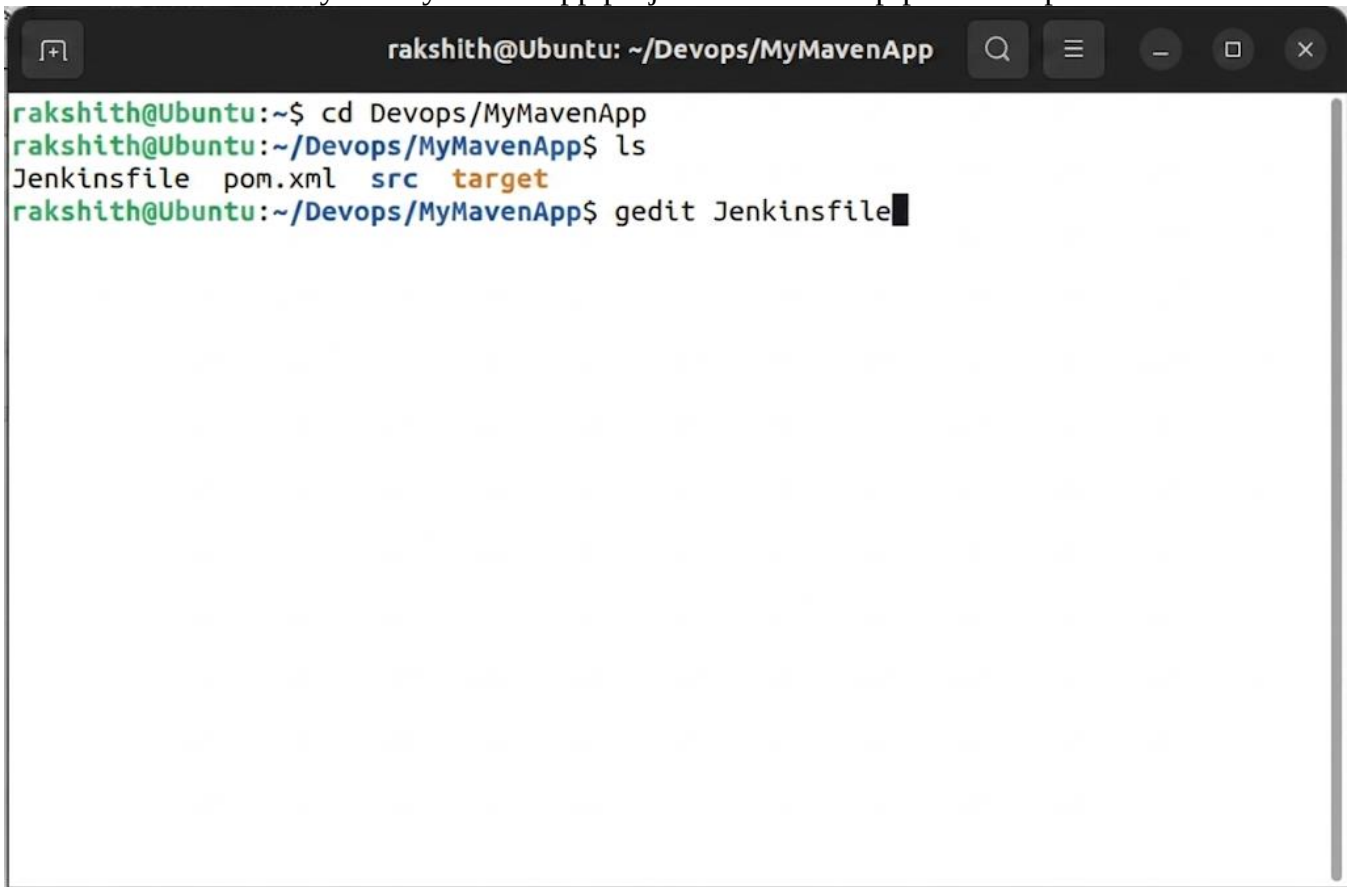
SCM: Git Enter repository URL and select GitHub credentials. Click Apply and Save.



The screenshot shows the Jenkins 'Configure' page for a job named 'MyMaven01App'. The left sidebar has a 'Configure' section with four tabs: 'General', 'Triggers', 'Pipeline' (selected), and 'Advanced'. The main content area is titled 'Pipeline' and has a subtitle 'Define your Pipeline using Groovy directly or pull it from source control.' Below this is a 'Definition' section with a dropdown menu set to 'Pipeline script from SCM'. Underneath is an 'SCM' section with a dropdown menu set to 'Git'. Below the 'SCM' section is a 'Repositories' section with a 'Repository URL' input field containing the value `https://github.com/rakshith31-hub/MyMavenApp.git`. Below the 'Repository URL' field is a 'Credentials' dropdown menu set to 'GitHub Credential' and a '+ Add' button. At the bottom of the 'Repositories' section is an 'Advanced' dropdown menu. At the bottom of the page are two buttons: 'Save' and 'Apply'.

Step 11: Create Jenkinsfile

Create a Jenkinsfile in your MyMavenApp project and add the pipeline script.

A terminal window titled 'rakshith@Ubuntu: ~/Devops/MyMavenApp' with standard window controls. The terminal shows the following commands and output:

```
rakshith@Ubuntu:~$ cd Devops/MyMavenApp
rakshith@Ubuntu:~/Devops/MyMavenApp$ ls
Jenkinsfile pom.xml src target
rakshith@Ubuntu:~/Devops/MyMavenApp$ gedit Jenkinsfile
```

Enter following code into jenkins file pipeline

```
{
  agent any // Use any available agent

  tools {
    maven 'Maven' // Ensure this matches the name configured in Jenkins
  }
  stages {
    stage('Checkout') {
      steps {
        git branch: 'master', url:
'https://github.com/rakshith31-hub/Mymavenapp.git'
      }
    }
  }
}
```

```

stage('Build')
{
    steps {
        sh 'mvn clean package' // Run Maven build
    }
}

stage('Test')
{
    steps {
        sh 'mvn test' // Run unit tests
    }
}

stage('Run Application') {
    steps {
        // Start the JAR application
        sh 'java -jar target/MyMavenApp-1.0-SNAPSHOT.jar'
    }
}

}

post {
    success {
        echo 'Build and deployment successful!'
    }
    failure {
        echo 'Build failed!'
    }
}
}

```

Step 12: Update pom.xml

Change source and target versions from 1.7 to 1.8.
Commit changes and push to your github repository.

Step 13: Build the Pipeline

Go to Jenkins and click Build Now.

Monitor Build Stages Use Stage View and Full Stage View to monitor execution.

Jenkins / All / MyMaven01App

MyMaven01App Add description

Stage View

Average stage times: (full run time: ~1min 24s)

	Declarative: Checkout SCM	Declarative: Tool Install	Checkout	Build	Test	Run Application	Declarative: Post Actions
#3 Apr 13 13:52 No Changes	3s	771ms	3s	55s	16s	1s	414ms
#2 Apr 13 13:46 1 commit	2s	469ms	2s	37s	26s	1s	301ms
#1 Apr 13 13:41 No Changes	2s	545ms	3s	54s	23s	1s	354ms
	5s	1s	4s	1min 13s failed	1s failed	748ms failed	588ms

Permalinks

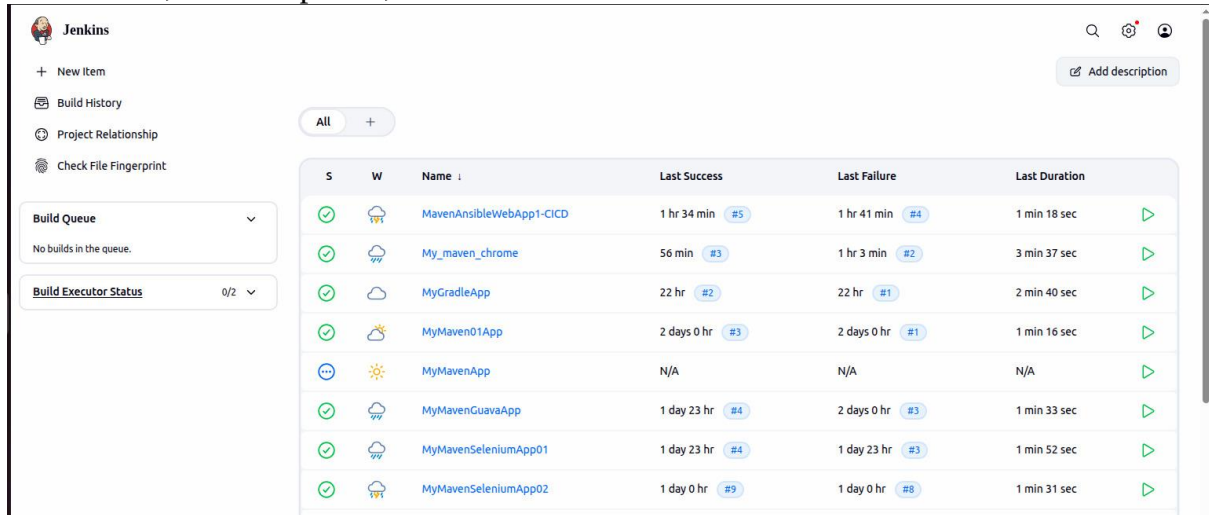
- [Last build \(#3\), 1 day 16 hr ago](#)
- [Last stable build \(#3\), 1 day 16 hr ago](#)
- [Last successful build \(#3\), 1 day 16 hr ago](#)
- [Last failed build \(#1\), 1 day 17 hr ago](#)

Gradle Pipeline:

Step 1: Create a New Pipeline Job

Go to Dashboard → + New Item.

Enter name, select Pipeline, and click OK.



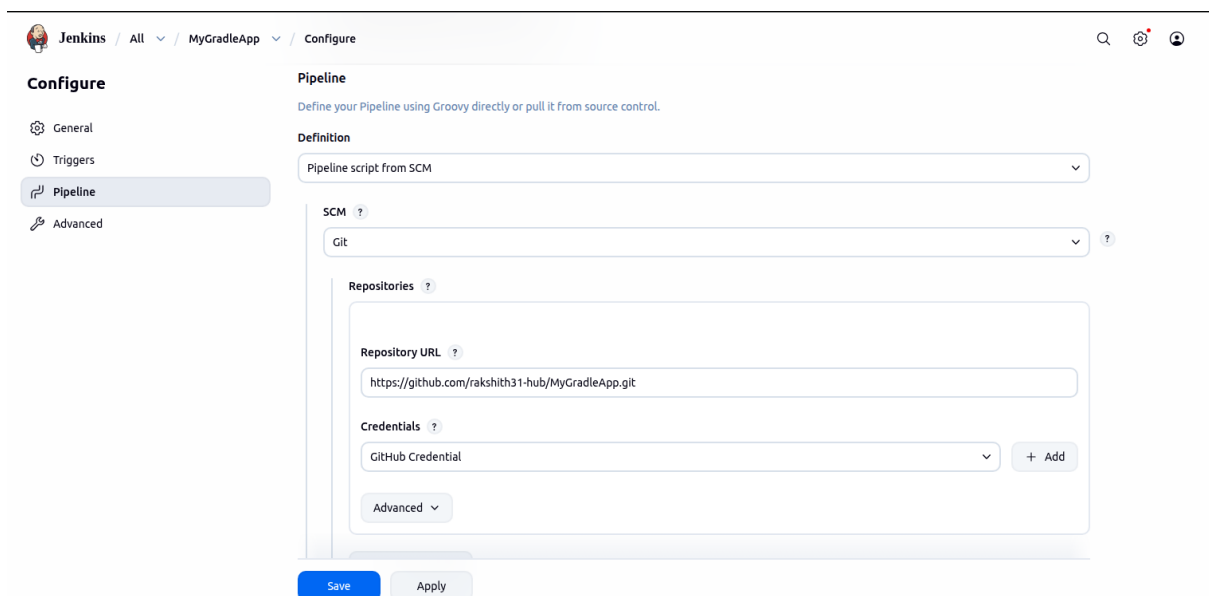
The screenshot shows the Jenkins Dashboard. On the left, there's a sidebar with 'New Item', 'Build History', 'Project Relationship', and 'Check File Fingerprint'. Below this, there are sections for 'Build Queue' (showing 'No builds in the queue.') and 'Build Executor Status' (showing '0/2'). The main area displays a table of pipeline jobs. The table has columns for 'S' (status), 'W' (weather icon), 'Name', 'Last Success', 'Last Failure', and 'Last Duration'. The jobs listed are:

S	W	Name	Last Success	Last Failure	Last Duration
✓	☁	MavenAnsibleWebApp1-CICD	1 hr 34 min #5	1 hr 41 min #4	1 min 18 sec
✓	☁	My_maven_chrome	56 min #3	1 hr 3 min #2	3 min 37 sec
✓	☁	MyGradleApp	22 hr #2	22 hr #1	2 min 40 sec
✓	☁	MyMaven01App	2 days 0 hr #3	2 days 0 hr #1	1 min 16 sec
⏸	☀	MyMavenApp	N/A	N/A	N/A
✓	☁	MyMavenGuavaApp	1 day 23 hr #4	2 days 0 hr #3	1 min 33 sec
✓	☁	MyMavenSeleniumApp01	1 day 23 hr #4	1 day 23 hr #3	1 min 52 sec
✓	☁	MyMavenSeleniumApp02	1 day 0 hr #9	1 day 0 hr #8	1 min 31 sec

Step 2: Configure Pipeline from SCM

Scroll down and select Pipeline script from SCM.

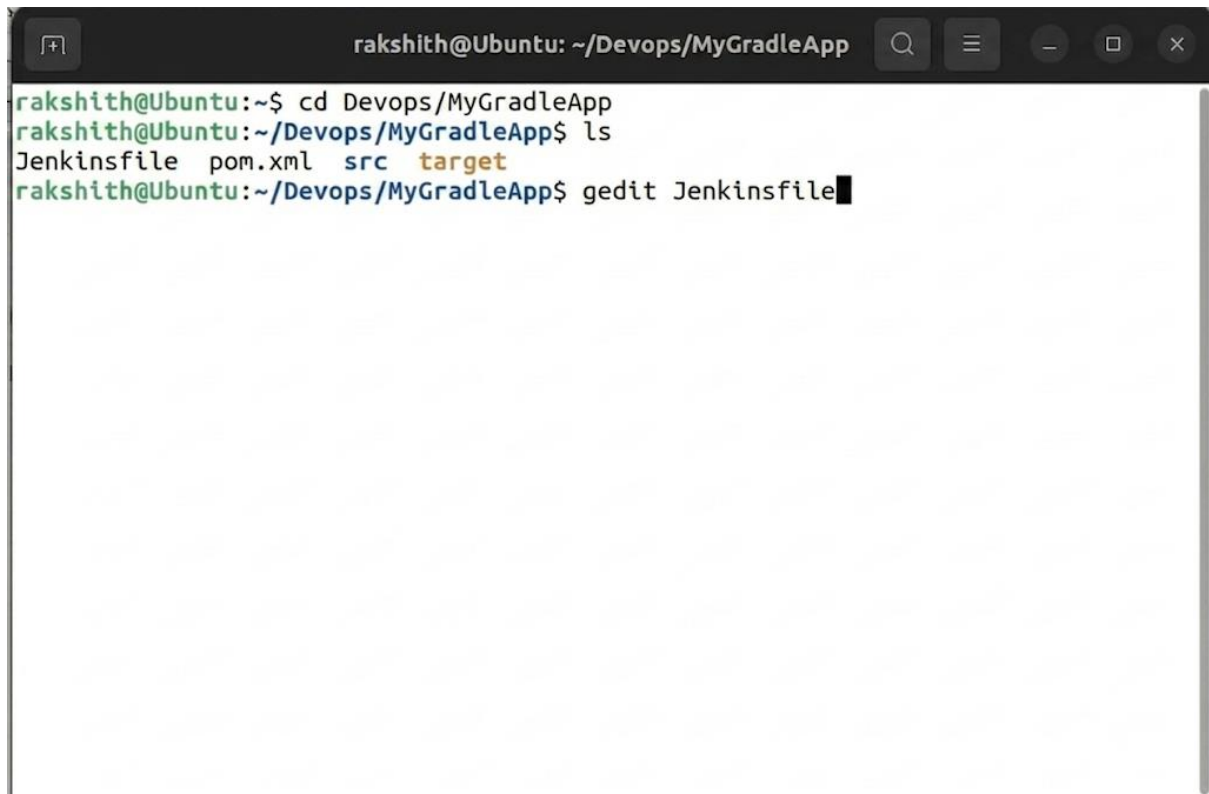
SCM: Git Enter repository URL and select GitHub credentials.



The screenshot shows the Jenkins Pipeline Configuration page. The left sidebar has 'Configure' at the top, followed by 'General', 'Triggers', 'Pipeline' (selected), and 'Advanced'. The main area is titled 'Pipeline' and has a subtitle 'Define your Pipeline using Groovy directly or pull it from source control.' Below this, there's a 'Definition' section with a dropdown menu set to 'Pipeline script from SCM'. Underneath, there's an 'SCM' section with a dropdown menu set to 'Git'. Below that, there's a 'Repositories' section with a 'Repository URL' field containing 'https://github.com/rakshith31-hub/MyGradleApp.git'. There's also a 'Credentials' dropdown menu set to 'GitHub Credential' and an '+ Add' button. At the bottom, there are 'Save' and 'Apply' buttons.

Step 3: Create Jenkinsfile:

Create a Jenkinsfile in your MyGradleApp project and add the pipeline script.

A terminal window titled 'rakshith@Ubuntu: ~/Devops/MyGradleApp'. The prompt is 'rakshith@Ubuntu:~\$'. The user enters 'cd Devops/MyGradleApp'. The prompt changes to 'rakshith@Ubuntu:~/Devops/MyGradleApp\$'. The user enters 'ls'. The output is 'Jenkinsfile pom.xml src target'. The user enters 'gedit Jenkinsfile'.

```
rakshith@Ubuntu:~$ cd Devops/MyGradleApp
rakshith@Ubuntu:~/Devops/MyGradleApp$ ls
Jenkinsfile pom.xml src target
rakshith@Ubuntu:~/Devops/MyGradleApp$ gedit Jenkinsfile
```

```
pipeline {
  agent any // Use any available agent

  tools {
    gradle 'Gradle' // Ensure this matches the name configured in Jenkins
    jdk 'JDK'
  }
  stages {
    stage('Checkout') {
      steps {
        git branch: 'master', url: 'https://github.com/harishbitcse82/MyGradleApp01.git'
      }
    }

    stage('Build') {
      steps {
        sh 'gradle build' // Run Gradle build
      }
    }

    stage('Test') {
      steps {
        sh 'gradle test' // Run unit tests
      }
    }

    stage('Run Application') {
```

```

    steps {
        // Start the JAR application
        sh 'gradle run'
    }
}

}

post {
    success {
        echo 'Build and deployment successful!'
    }
    failure {
        echo 'Build failed!'
    }
}
}

```

Step 4: Build the Pipeline

Go to Jenkins and click Build Now. Monitor Build Stages Use Stage View and Full Stage View to monitor execution.

The screenshot shows the Jenkins web interface for a pipeline named 'MyGradleApp'. The left sidebar contains navigation links: Status, Changes, Build Now, Configure, Delete Pipeline, Full Stage View, Stages, Rename, and Pipeline Syntax. The main area displays the 'Stage View' for the pipeline. At the top, there's a status bar with a green checkmark and the pipeline name 'MyGradleApp', along with an 'Add description' button. Below this, the 'Stage View' section shows a table of stages and their durations. The stages are: Declarative: Checkout SCM (6s), Declarative: Tool Install (2s), Checkout (4s), Build (1min 28s), Test (9s), Run Application (9s), and Declarative: Post Actions (483ms). The table also shows the average stage times and the full run time (~2min 40s). Below the table, there's a 'Permalinks' section with links to the last build, last stable build, last successful build, last failed build, last unsuccessful build, and last completed build. On the left, there's a 'Builds' section showing a list of builds with a filter input and a dropdown menu.

	Declarative: Checkout SCM	Declarative: Tool Install	Checkout	Build	Test	Run Application	Declarative: Post Actions
Average stage times: (full run time: ~2min 40s)	6s	2s	4s	1min 28s	9s	9s	483ms
#2 Apr 14 15:25 No Changes	6s	2s	4s	1min 28s	9s	9s	483ms
#1 Apr 14 15:15 No Changes							

Permalinks

- [Last build \(#2\), 15 hr ago](#)
- [Last stable build \(#2\), 15 hr ago](#)
- [Last successful build \(#2\), 15 hr ago](#)
- [Last failed build \(#1\), 15 hr ago](#)
- [Last unsuccessful build \(#1\), 15 hr ago](#)
- [Last completed build \(#2\), 15 hr ago](#)

Builds

14 April 2026

Filter [/]

- ✓ #2 15:25
- ✗ #1 15:15